Universität Konstanz

Introduction to Natural Language Processing (NLP) and Large Language Models (LLMs)

Jun.-Prof. Dr. Miriam Schirmer

Konstanz, 29.10.2026

Outline for Today

Session 2: Text as Data

1. Recap: What is NLP?
2. Readings: text as data
3. Words & tokens, pre-processing
4. Regular expressions
5. Lexicons
6. Admin

Natural Language Processing

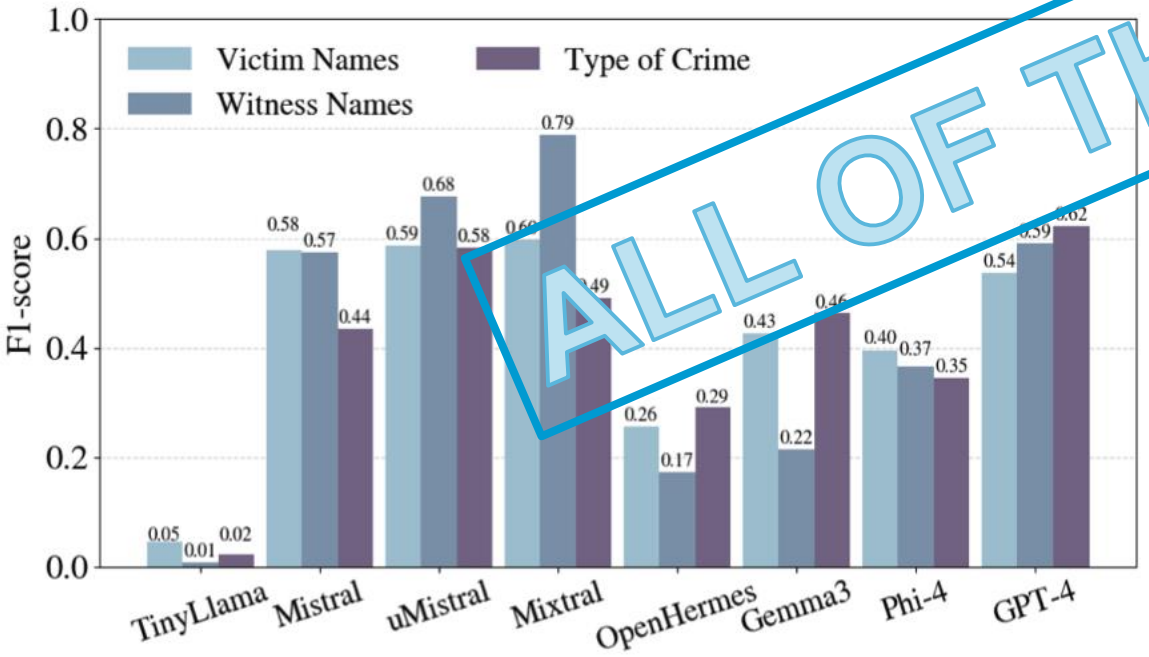


Human language, the most **natural** to us.

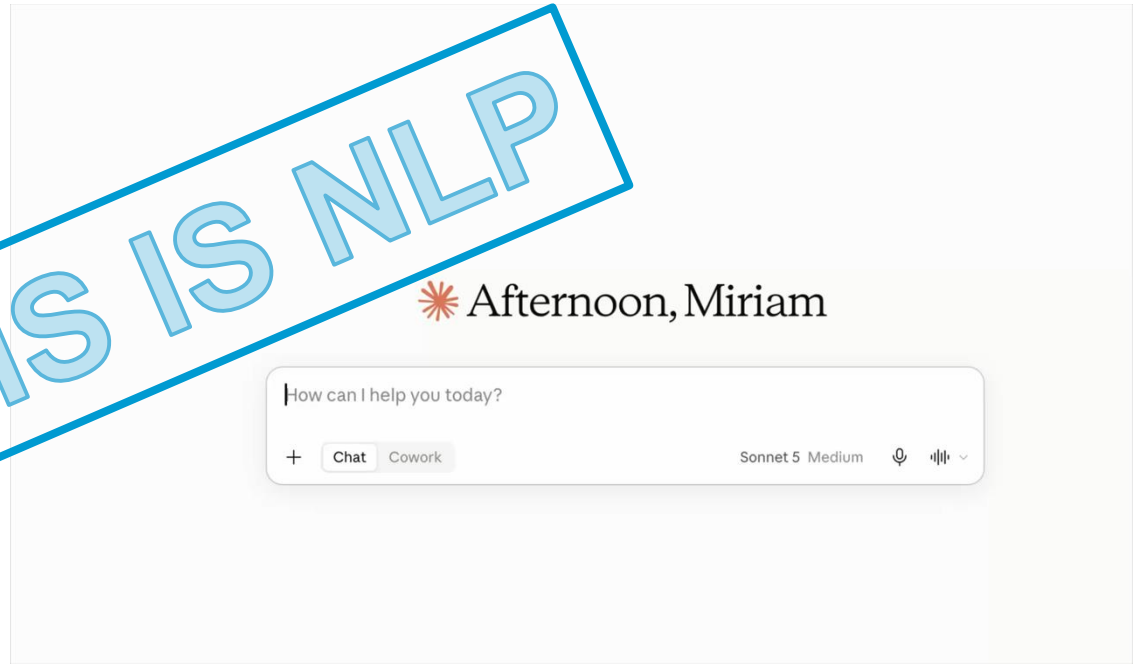


Algorithms! But also Maths, coding, engineering, hardware, investments…



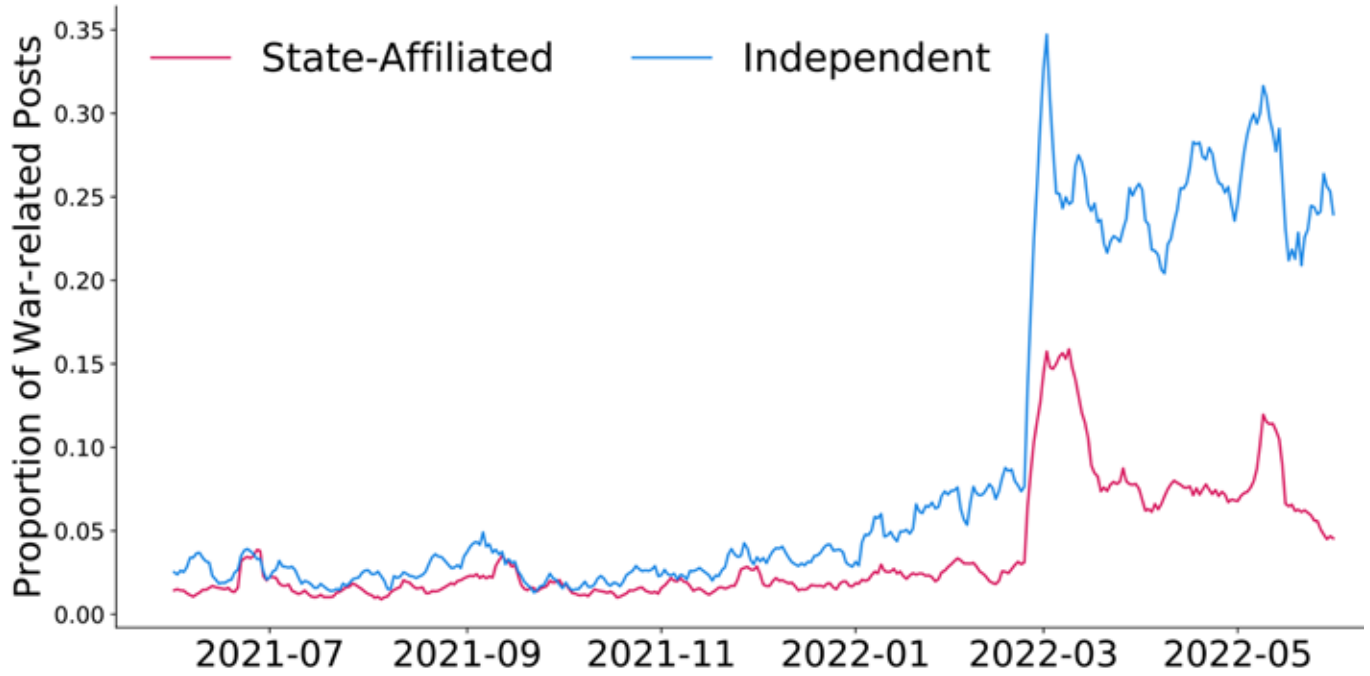


ALL OF THIS IS NLP



Counting Words

Proportion of posts mentioning war-related words

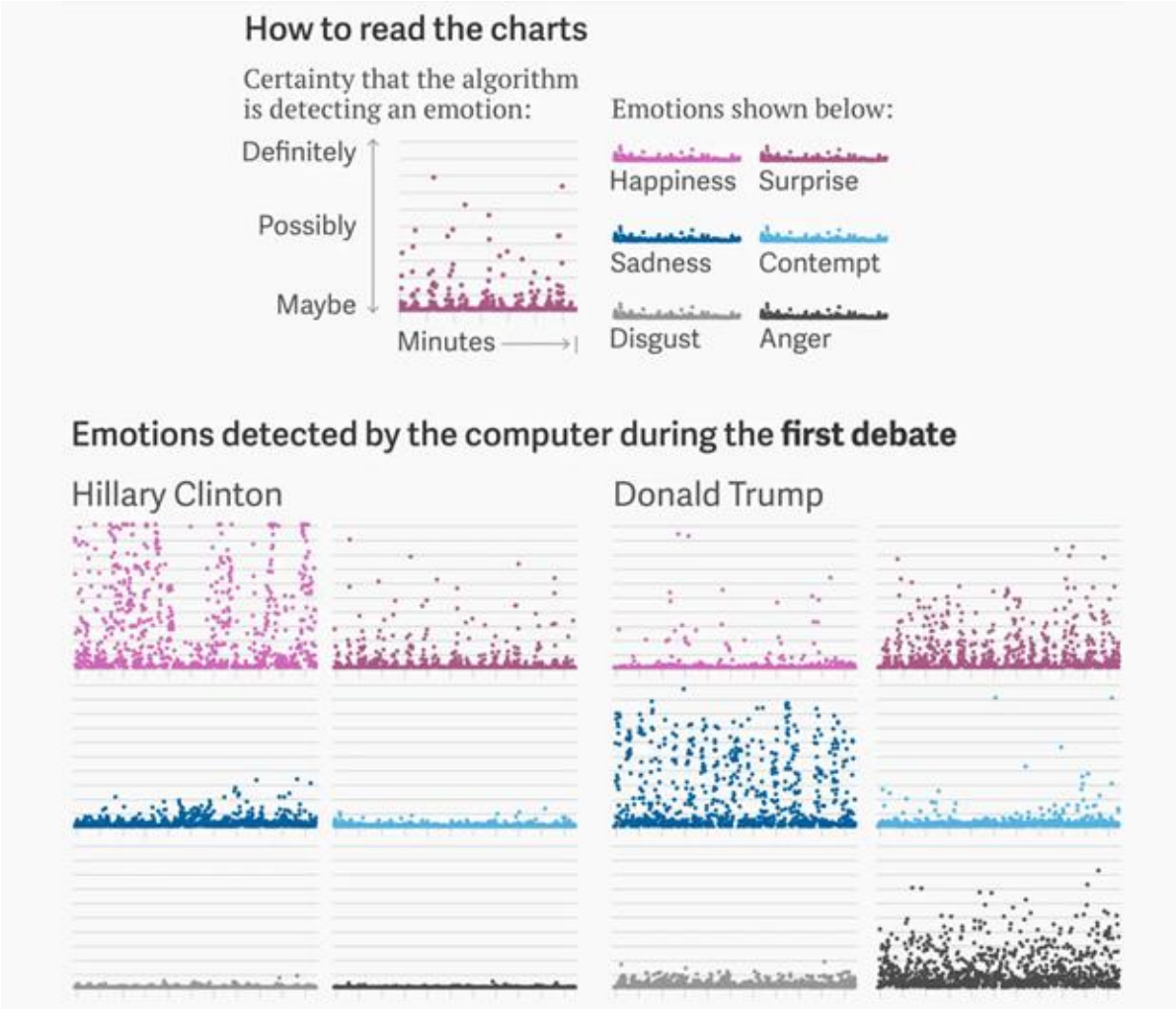


Park, C. Y., Mendelsohn, J., Field, A., & Tsvetkov, Y. (2022). Challenges and Opportunities in Information Manipulation Detection: An Examination of Wartime Russian Media. EMNLP Findings.

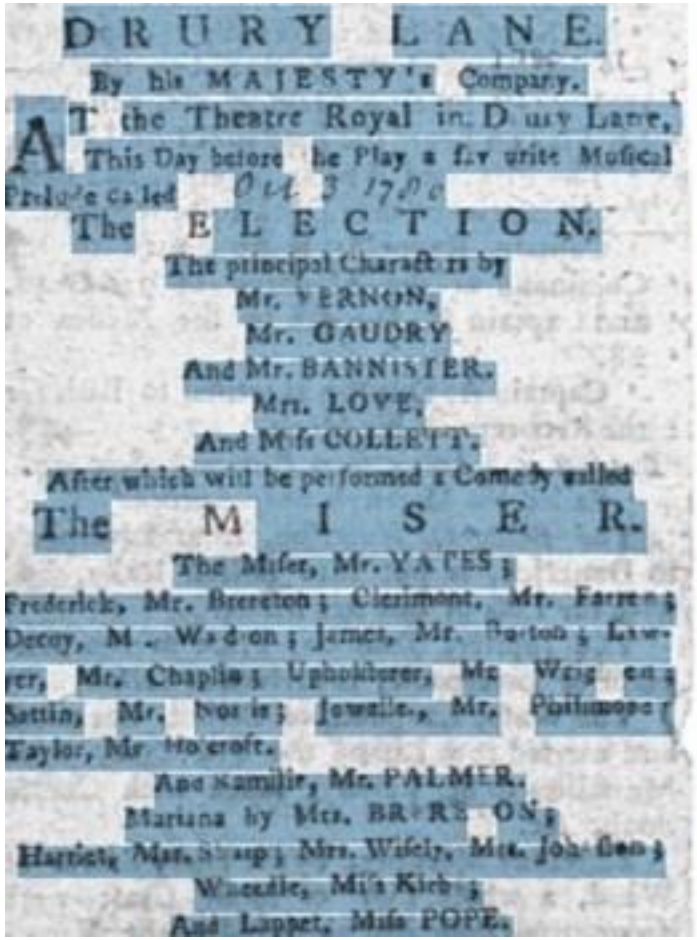
Lexicons (groups of words, e.g. emotions)

A computer watched the debates. It thought Clinton was happy and Trump was angry and quite sad

https://qz.com/810092/a-computer-watched-the-debates-and-thought-clinton-happy-trump-angry-sad



Optical Character Recognition



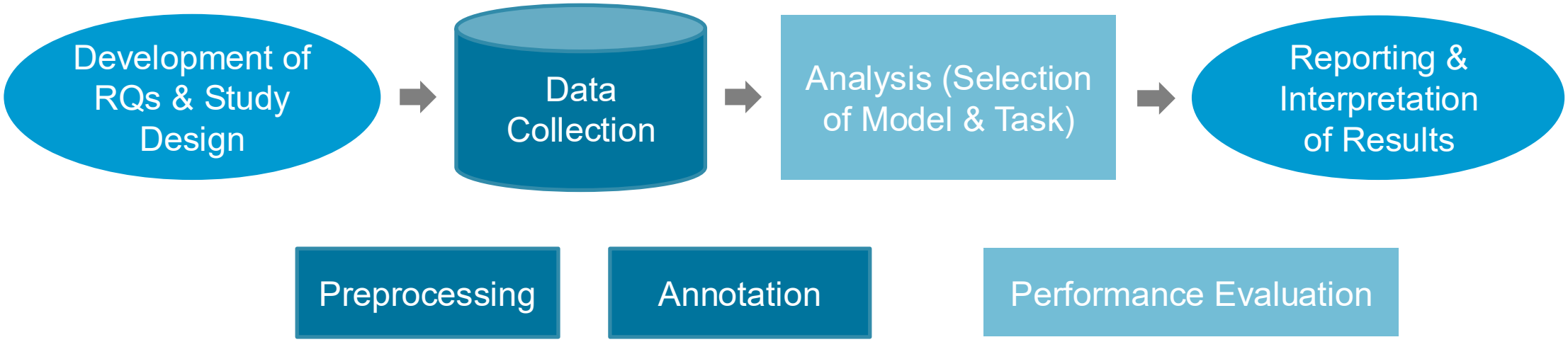
DRURY LANE.
By his MAJESTY's Company.
AT the Theatre Royal in Drury Lane,
This Day before he Play a fav'rite Musical
Prelude called
The ELECTION.
The principal Characters by
Mr. VERNON,
Mr. GAUDRY
And Mr. BANNISTER.
Mrs. LOVE,
And Mrs COLLETT.
After which will be performed a Comedy called
The MISER.
The Miser, Mr. YATES;
Frederick, Mr. Brereton; Clerimont, Mr. Farren;
Decoy, Mr. Wadon; James, Mr. Parton; Law-
yer, Mr. Chaplin; Upholderer, Mr. Wrighten;
Sattin, Mr. Norris; Jeweller, Mr. Poitmore;
Taylor, Mr. Holcroft.
And Ramillie, Mr. PALMER.
Mariana by Mrs. BRERETON;
Harriet, Mrs. Sharp; Mrs. Wilely, Mrs. Johnston;
Woodale, Miss Kirby;
And Lappet, Miss POPE.

DRURY LANE.
By his MAJESTY' , Compar.y
AT the Theatre Roys. . in
This Day before he Play a f.r uri
PtcWt ca led 0 vt 3 I? ftp
The ELECTION:
The principal Charaer n b,
Mr. VERNON,
Mr. C.UDRY
And Mr. BANNISTER.
Mrs. LOVE,
And Mfs COLLETT.
ASter which will be pe: formed s
The MISER.
The Miter, Mr. YA TES }
Frederick, Mr. Brereton; Clerimon
Decoy, M . W a d o n ; James, Mr.
Mr. Chap.is ; Upholderer, Mr Wng
Sattin, Mr. N o i « ; Jevrei.e.,
Ta) lor, Mr Ho croft.
ABO Ramjlif, Mr. PALMER.
Mariana hy Mrs. BR-RE O.V;
Harriet, M.s. S atp ; Mrs. Wife].
W..eidle, Mi's Kirb ;
Aad Lappet, Mils POPE.
To which will be added a Pernaj.
Part.)



https://ocr.northeastern.edu/

NLP Pipeline



Let’s talk about the readings!

Let’s talk about the readings

In pairs (5 min), then together

- **Jurafsky & Martin, ch. 2:** one thing you learned or found surprising about words and tokens
- **Benoit (2020):** what does it mean to treat text “as data” rather than “as text”?
- **Both:** what is lost when a text becomes a table of numbers, and why might it be worth it?

Text as text vs. text as data

Benoit (2020): the key difference is an intermediate step

- **Text as text:** we read and interpret the text directly (e.g., discourse analysis)
- **Text as data:** we first extract features from the text (e.g., word counts) and then analyze these features

"[I]n order to treat text as data rather than text as text, we must destroy the immediate interpretability of source texts, but for the higher purpose of enabling more systematic, larger-scale inference from their stylised features." (p. 7)

What we are usually after: latent characteristics

- Ideology, policy preferences or personality cannot be observed directly
- What political actors say or write is an observable implication of these inner states

A map of approaches to political text

Approach	Method	Target of concern
Literary	Discourse analysis	Meaning, rhetoric, power relations, hidden biases, symbolism
Qualitative	Thematic analysis, content analysis	Topics, positions
Hybrid quantitative	Dictionary analysis	Affect
Purely quantitative	Statistical summary, machine learning	Authorship, intent, similarity, events

From top to bottom: more automation, and the text is treated less “as text” and more “as data”.

Simplified from Benoit (2020), Table 26.1

Stages of analysing text as data

1. Selecting texts and defining the corpus
2. Converting the texts into a common electronic format
3. Defining documents and choosing the unit of analysis
4. Defining and refining features
5. Converting textual features into a quantitative matrix
6. Analysing the (matrix) data using an appropriate statistical procedure
7. Interpreting and reporting the results

Benoit (2020), Table 26.2. Today: steps 3 to 5. Later in the course: steps 6 and 7.

Your turn: design a text-as-data study

In pairs (5 min)

Research question: do populist parties use simpler language than other parties?

- **Corpus:** which texts would you collect, and from where?
- **Documents:** what is one document: a speech, a manifesto, a sentence, a social media post?
- **Features:** what would you count?
- **Pitfalls:** which texts might be missing from your corpus, and why?

Kittel (2025) asks a similar question: you will see the paper's answer in the session on 12 November.

Words!

I'll start by asking you: what is a word?

There’s actually no consensus among linguists

- Is it based on how language is written (orthographic)?
- How we break up continuous sound into discrete pieces (phonetics/phonology)?
- Defined by the grammar of a language (morphology/syntax)?

Basic characteristics: **words carry meaning, can be used on their own, and are uninterruptible**

Where defining words gets weird (in English)

- Should punctuation marks count as words?
- What about **filled pauses** and **fragments** in transcribed speech?
  - *"I do **uh main-** mainly business data processing"* [example from SLP]
- Are contractions such as ***I'm*** one word or two?
  - And if it's two words, is the second word just ***'m*** or ***am***?
- What about acronyms and abbreviations (***U.S.A***)?
- What about ***New York*** or ***threw up***?
  - There are spaces but you need both pieces to understand the concept

How can this affect our text analysis and results?

Tokenization is everything

In NLP, **tokens are our primary unit of analysis and used as inputs for all models**

- Important considerations: how well does our tokenization method capture patterns in the data; how tokenization affects performance on downstream tasks
- The performance of even the biggest fanciest LLMs entirely rests on robust tokenization
- Data-driven algorithms (byte-pair encoding)

A little bit of…vocabulary

Token: basic unit of text for computational language processing

Document: the larger text that gets broken down into tokens

Corpus (pl. corpora): a set of documents

Vocabulary: the set of distinct tokens in a corpus

Rule-based tokenization

- Use **predefined rules** to tokenize a document
- Ex: remove punctuation, unless it's in a website url, a known abbreviation, or a hyphen within a word.
- Ex: keep multiword expressions as single tokens if they are known geopolitical entities ("New York", "United Kingdom", "Las Vegas")
- NLTK and SpaCy are Python packages that are good options for rule-based tokenization
- Packages like SpaCy allow you to define custom tokenization rules

Subword Tokenization (Byte Pair Encoding)

- Instead of dealing with these thorny questions about “words” & “morphemes”, **modern NLP approaches learn how to tokenize from the data itself.**

- These tokenizers have two parts:
  1. The token **learner**: induces a *vocabulary* from a *corpus*
  2. The token **segmenter**: tokenizes a new document according to the learned vocabulary

- Byte-Pair Encoding (BPE) is the most common subword tokenization approach
- It builds a subword vocabulary by repeatedly merging the most frequent pair of neighbouring symbols in a training corpus
- Common words end up as single tokens, and rare or unknown words are split into smaller known pieces
- Example: unhappiness → un + happi + ness.

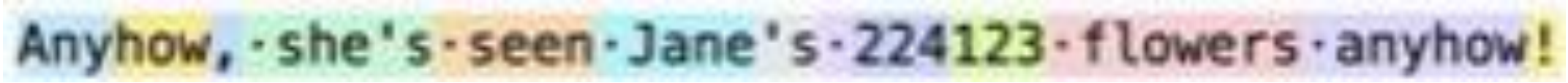
Language models process tokens, not words

A **token** is a word or word part.

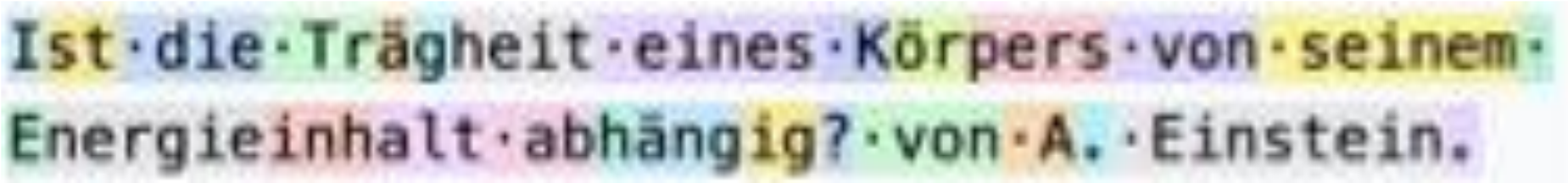
First LLM step is converting text into tokens

- Via tokenization algorithms like **BPE**

English tokens correspond roughly to words:



German tokens tend to be smaller than words



Questions about words and tokens?

Pre-processing steps

Minimize Variations

* I've been in New York in 2011, but didn't like it. I preferred LOS ANGELES. *

Pre-processing steps

Formatting removal

Remove any explicit formatting the text my have.

Example: HTML formatting.

```
<div id=«text»> I’ve been in New
York in 2011, but didn’t like it.
I preferred LOS ANGELES.</div>
```



I’ve been in New York in 2011, but didn’t like it. I preferred LOS ANGELES.

Pre-processing steps

Formatting removal

Remove any explicit formatting the text my have.

Example: HTML formatting.

```
<div id=«text»> I’ve been in New
York in 2011, but didn’t like it.
I preferred LOS ANGELES.</div>
```



I’ve been in New York in 2011, but didn’t like it. I preferred LOS ANGELES.

Example: map everything to lower case.

I’ve been in New York in 2011, but didn’t like it. I preferred LOS ANGELES.



i’ve been in new york in 2011, but didn’t like it. i preferred los angeles.

Pre-processing steps

Numbers Normalization

Map numerical expressions to a canonical form to reduce variation:

i’ve been in new york in 2011, but didn’t like it.
i preferred los angeles.



i’ve been in new york in 0000, but didn’t like it. i preferred los angeles.

Pre-processing steps

Numbers Normalization

Map numerical expressions to a canonical form to reduce variation:

i’ve been in new york in 2011, but didn’t like it.
i preferred los angeles.



i’ve been in new york in 0000, but didn’t like it. i preferred los angeles.

Join collocations

Merge multi-word expressions that function as a single unit into one token to preserve their meaning:

i’ve been in new york in 0000, but didn’t like it. i preferred los angeles.



i’ve been in new_york in 0000, but didn’t like it. i preferred los_angeles.

Pre-processing steps

Lemmatization

Map each word to the corresponding **«lexeme»**, i.e. the corresponding dictionary entry without any morophological variation:

i’ve been in new_york in 0000, but didn’t like it. i preferred los_angeles.



i have be in new york in 0000, but do not like it. i prefer los_angeles.

Pre-processing steps

Stopwords removal

Remove words that are not relevant, so called *stopwords*.

i have be in new_york in 0000, but do not like it. i prefer los_angeles.

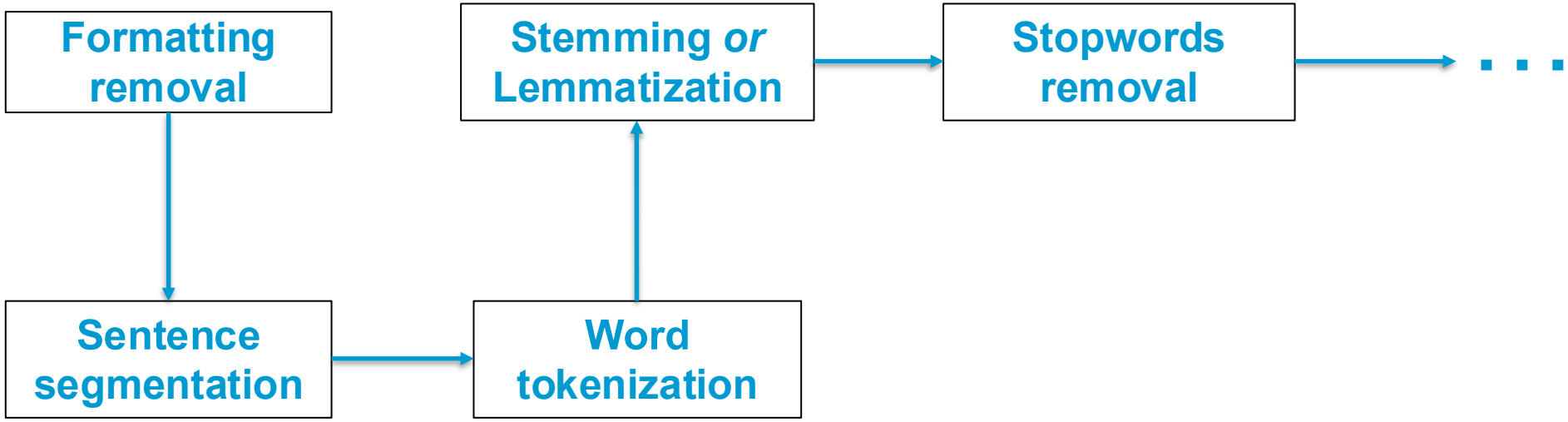


new_york 0000, not like. prefer los_angeles

a	also	an	and	as	at	be	but	by
can	could	do	for	from	go			
have	he	her	here	his	how			
i	if	in	into	it	its			
my	of	on	or	our	say	she		
that	the	their	there	therefore	they			
this	these	those	through	to	until			
we	what	when	where	which	while	who	with	would
you	your							

- Requires a **language-specific list** of stopwords.
- To be performed **after lemmatization**.

Pre-processing steps



Luckily, we can use specialized tools to **perform text pre-processing automatically**!

Why / how can pre-processing impact your data and analysis?

Pre-processing choices matter

Benoit (2020) on common pre-processing steps

- **Lower-casing and stemming** treat taxes, tax, taxation, taxing, taxed and taxable as one feature: tax
- **Stopword lists:** there is no list of words that is useless for every analysis. The pronoun “her” is clearly partisan in US Senate debates on abortion (Monroe et al. 2008).
- **n-grams** keep phrases that change the meaning: command economy vs. market economy

Always ask: does this step remove something my research question needs?

Regular expressions (“regex”)

A formal language for searching for patterns in text strings (= sequence of characters)

- Super useful for:
  - Cleaning messy unstructured text data
  - Extracting keywords of interest from text
  - Querying databases
- Built-in Python package: `re`
  - https://docs.python.org/3/howto/regex.html
- Use Python raw string notation: `r"regex"`



https://www.reddit.com/r/ProgrammerHumor/comments/c9e0n4/regex_is_so_easy/

Regular Expressions: Disjunctions

Letters inside square brackets []

Ranges using the dash `[A-Z]`

Pattern	Matches
r"[mM]ary"	Mary or mary
r"[1234567890]"	Any one digit

Pattern	Matches	
`r"[A-Z]"`	An upper case letter	`Drenched Blossoms`
`r"[a-z]"`	A lower case letter	`my beans were impatient`
`r"[0-9]"`	A single digit	`Chapter 1: Down the Rabbit Hole`

Regular expressions: more patterns

Pattern	Meaning	Example
`r"o+"`	one or more of the previous character	so, sooooo
`r"o{2,}"`	two or more	soo, sooooo (not so)
`r"cat|dog"`	either … or	cat, dog
`r"\S+"`	one or more non-space characters	https://example.com
`r"(\w)\1"`	a group, and the same character again	the oo in sooooo
`r"[i1]"`	one of these characters	idiot, id1ot

Re-use: re.findall(pattern, text), re.sub(pattern, replacement, text)

Regular Expressions: Word Boundaries

- `r"\b"` is a word boundary.
- Ex: My **cat** is a **cat**astrophe. He acts like a wild**cat**.
- `r"\bcat\b"` *only* matches the full word “cat”
- `r"cat\b"` matches the characters ***cat*** in “cat” and “wildcat” but *not* “catastrophe”



Tips for regexes

Iterate on your regex using an online regex tester.

https://regex101.com/

Be sure to test tricky edge cases where your regex *should* and *should not* match the string.

Lexicons

From words to lexicons

We've talked about:

- Identifying (& counting) words when you know what words you're interested in
- Comparing two corpora based on word statistics, when you don't know what words you're interested in

Lexicons are useful when we know what broader **concepts** we're interested in

- Sentiment and emotions are by far the most common concepts
- But you can use lexicons for any concept!

What's a lexicon?

- Lexicons are **sets of words (or phrases)** that capture a particular **concept of interest** (e.g. *positive sentiment, moral disgust, populism*)
- Some lexicons assign scores to words to represent the intensity or level of association with the concept.
  - e.g. "love" might have a higher score than "like" for positive sentiment
- A note on terminology:
  - We're going to use the term *lexicon*, which is preferred among computational linguists.
  - Other researchers prefer *dictionary* but this gets confusing when we have the Python dictionary data structure.
  - But keep this in mind when looking for related literature in non-CL fields!

Dictionary analysis (Benoit 2020)

- **Key:** a concept, e.g. positive emotion (posemo)
- **Values:** patterns that count as the key, e.g. kind, kindly, kindn* (the * matches kindness, kindnesses)

The challenge: calibrating the patterns

- **Too broad:** kind* also matches Kindle, the e-book reader (false positives)
- **Too narrow:** kind alone misses kindly (false negatives)
- **Polysemy:** in US State of the Union addresses, about 95% of 318 uses of “kind” are the noun (“a kind of …”), only about 4% express a positive emotion

Coming up on 12 November: Gründl (2022) measures populist communication with a dictionary.

Example Lexicon: sentiment

Positive	admire, amazing, assure, celebration, charm, eager, enthusiastic, excellent, fancy, fantastic, frolic, graceful, happy, joy, luck, majesty, mercy, nice, patience, perfect, proud, rejoice, relief, respect, satisfactorily, sensational, super, terrific, thank, vivid, wise, wonderful, zest
Negative	abominable, anger, anxious, bad, catastrophe, cheap, complaint, condescending, deceit, defective, disappointment, embarrass, fake, fear, filthy, fool, guilt, hate, idiot, inflict, lazy, miserable, mourn, nervous, objection, pest, plot, reject, scream, silly, terrible, unfriendly, vile, wicked

Figure 22.3 Some words with consistent sentiment across the General Inquirer (Stone et al., 1966), the MPQA Subjectivity lexicon (Wilson et al., 2005), and the polarity lexicon of Hu and Liu (2004).

Jurafsky & Martin, *ch. "Lexicons for Sentiment, Affect, and Connotation"*

Example: Linguistic Inquiry and Word Count (LIWC)

Positive Emotion	Negative Emotion	Insight	Inhibition	Family	Negate
appreciat*	anger*	aware*	avoid*	brother*	aren't
comfort*	bore*	believe	careful*	cousin*	cannot
great	cry	decid*	hesitat*	daughter*	didn't
happy	despair*	feel	limit*	family	neither
interest	fail*	figur*	oppos*	father*	never
joy*	fear	know	prevent*	grandf*	no
perfect*	griev*	knew	reluctan*	grandm*	nobod*
please*	hate*	means	safe*	husband	none
safe*	panic*	notice*	stop	mom	nor
terrific	suffers	recogni*	stubborn*	mother	nothing
value	terrify	sense	wait	niece*	nowhere
wow*	violent*	think	wary	wife	without

Figure 22.6 Samples from 5 of the 73 lexical categories in LIWC (Pennebaker et al., 2007). The * means the previous letters are a word prefix and all words with that prefix are included in the category.

2300 words
>70 classes, including sentiment and emotions

Jurafsky & Martin, *ch. "Lexicons for Sentiment, Affect, and Connotation"*

Example: Ideologies and Policy Issues

Table A2. Dictionary

Dictionary	Dutch words	Translation
Conservatism	christ*; geloof; gezin; kerk; normen; porn*; seks*; waarden	christ*; belief; family; church; norm; porn*; sex*; values
Environment	ecol*; groene*; klimaat*; milieu*; opwarming	ecol*; green*; climate*; environment*; heating
Immigration	marok*; turk; allocht*; asiel*; halal*; hoofddoek*; illega*; immigr*; islam*; koran; moslim*; vreemd*	moroc*; turk; allocht*; asylum*; halal*; scarf*; illega*; immigr*; islam*; koran; muslim*; foreign*

Ideologies - Pauwels (2011) Measuring Populism: A Quantitative Text Analysis of Party Literature in Belgium. Journal of Elections, Public Opinion and Parties 21(1): 97-119

When would you trust a dictionary count, and when would you rather read the texts?

Wrap-Up: Advantages and Disadvantages of Lexicons

Cost-efficient	Don’t account for polysemy (same word with different meanings)
Fast	Don’t account for document context
Easy to use	Potential sparsity of signal
Interpretable	Representativeness of concept

Paper presentations

Sign-up this week

- Individually or in pairs, one applied paper from the paper sessions (pick one, max 2 presentations per session, first come first served)
- **Would someone like to work in a pair but doesn't know anyone?**

Your presentation covers

- The research question and its relevance; the data (what texts, from where, how many)
- The method in your own words, linked to the theory session of the block; the main findings
- The limitations (would you trust this measurement?) and two or three questions for the discussion

If there's something in the paper you don't understand, ask me! Maybe it's not relevant for the presentation.

Before next week (5 November)

- **5 November: coding session.** Bring a laptop: we work with a notebook on pre-processing, bag of words and dictionaries in Google Colab.
- **Google account:** create one (a separate account just for this course is fine) and open colab.research.google.com once

- All readings are linked in the syllabus on GitHub.

Do not edit
How to change the design



How did you find the level of this class?

ⓘ The Slido app must be installed on every computer you're presenting from

slido

Do not edit
How to change the design



Audience Q&A

ⓘ The Slido app must be installed on every computer you're presenting from

slido

Universität Konstanz

Questions?
Thank you!

Jun.-Prof. Dr. Miriam Schirmer
Political and Administrative Science

miriam.schirmer@uni-konstanz.de

Credits: Parts of this slide deck have been adapted from slides created by Jurafsky & Martin, Debora Nozza, and Julia Mendelsohn.

Appendix

Regular expressions

The pattern

```
r"Buttercup"
```

matches the substring Buttercup in any string, like the string

```
I'm called little Buttercup
```

Regular Expressions: Negation in Disjunction

Carat as first character in [] negates the list
Note: Carat means negation only when it's first in []
Special characters (., *, +, ?) lose their special meaning inside []

Pattern	Matches	Examples
r"[^A-Z]"	Not upper case	Oyfn pripetchik
r"[^Ss]"	Neither 'S' nor 's'	I have no exquisite reason"
r"[^.]"	Not a period	Our resident Djinn
r"[e^]"	Either e or ^	Look up ^ now

Wildcard

- The period means "any character"
- `r"."` matches anything
- `r".*"` matches any sequence of 0 or more of anything
- `r"?"` optionally matches the previous character
  - `r"cats?"` matches both *cat* and *cats*

Regular Expressions: Convenient aliases

Pattern	Expansion	Matches	Examples
r"\d"	[0-9]	Any digit	Fahreneit 451
r"\D"	[^0-9]	Any non-digit	Blue Moon
r"\w"	[a-ZA-Z0-9_]	Any alphanumeric or _	Daiyu
r"\W"	[^\w]	Not alphanumeric or _	Look!
r"\s"	[\r\t\n\f]	Whitespace (space, tab)	Look␣up
r"\S"	[^\s]	Not whitespace	Look up

The iterative process of writing regex's

Find me all instances of the word “the” in a text.

`the`

Misses capitalized examples

`[tT]he`

Incorrectly returns `other` or `Theology`

`\b[tT]he\b`